

1 Task 1

Gauss_Elim_File.py ×ours.py ×main.py ×

```
1 import numpy as np
2 import sys
3 import matplotlib.pyplot as plt
4
5 # Input the number of finite elements and boundary conditions
6 ##
7 Ne = int(input("Enter the number of elements: ")) # number of finite elements
8 y0 = np.zeros(1)
9 y1 = np.zeros(1)
10 y0[0] = input("Input the BC at x=0: ") # BC at x=0
11 y1[0] = input("Input the BC at x=1: ") # BC at x=1
12 print("Input the quadrature rule type:") # quadrature rule type
13 print(" 1. Trapezoidal rule")
14 print(" 2. Simpson's rule")
15 quad_type=int(input())
16 ##
17 # Input the solver type
18 ##
19 print("Input the solver type:")
20 print(" 1. Direct solver (Gaussian elimination)")
21 print(" 2. Jacobi iteration")
22 sol_type=int(input())
23 if int(sol_type==2):
24     epst = float(input("Input the solution tolerance: "))
25
26 ##
27 # Calculate the number of nodes and the element size
28 ##
29 N = Ne + 1 # number of nodes
30 h = 1 / Ne # element size
31 ##
32 # Initialize the global matrix and the right-hand side vector
33 ##
34 A = np.zeros(N) # coefficient matrix main diagonal
35 B = np.zeros(N - 1) # coefficient matrix sub-diagonal
36 C = np.zeros(N - 1) # coefficient matrix super-diagonal
37 f = np.zeros(N) # right-hand side vector
38 ##
39 # Assemble the element matrices and right-hand sides
40 ##
41
42 def Numerical_Quadrature(e,i,quad_type,h):
43
44
45 for e in range(Ne): #e is element no. # loop over finite elements
46     Ae = np.zeros([2,2]) # element coefficient matrix
47     fe = np.zeros([2]) # element right-hand side
48     for i in range(2): # loop over the nodes in an element
49         ## ----- ##
50         ## HERE IS THE CALL TO THE NUMERICAL QUADRATURE FUNCTION THAT YOU NEED TO WRITE ##
51         fe[i]=Numerical_Quadrature(e,i,quad_type,h); # eleemt right-hand side vector contribution
52         ## ----- ##
53         for j in range(2): # loop over nodes in an element
54             Ae[i,j]=(-1)**(i+j)/h # element matrix contribution
55         ##
56     # Impose the boundary conditions
57     ##
```

1. Establishing Number of Element subdivisions

2. Establishing the global matrix and the right hand side vector from equations 13 a and 13 b in the CW document.

3. Using the trapezoidal rule to calculate each 'f' vector for every associated element.

Gauss_Elim_File.py ×ours.py ×main.py ×

```
55  ##
56  #   Impose the boundary conditions
57  ##
58  if e==0:                                # the first element at x=0
59      Ae[0,0]=1
60      Ae[0,1]=0
61      fe[0]=y0[0];
62      fe[1]=fe[1]-Ae[1,0]*y0[0]          # y=y0 at x=0;
63      Ae[1,0]=0                          # modify the second equation
64  if e==Ne-1:                             # the last element at x=1
65      Ae[1,1]=1
66      Ae[1,0]=0
67      fe[1]=y1[0]
68      fe[0]=fe[0]-Ae[0,1]*y1[0]          # y=y1 at x=1
69      Ae[0,1]=0                          # modify the second equation
70
71  ##
72  #   Transfer the elemnt matrix and the element right-hand side to global data structures
73  ##
74  A[e]=A[e]+Ae[0,0]                      # diagonal element Ae[0,0]
75  A[e+1]=A[e+1]+Ae[1,1]                  # diagonal element Ae[1,1]
76  B[e]=Ae[1,0]                          # sub diagonal element Ae[1,0]
77  C[e]=Ae[0,1]                          # super diagonal element Ae[0,1]
78  f[e]=f[e]+fe[0]                       # rhs element fe[0]
79  f[e+1]=f[e+1]+fe[1]                   # rhs element fe[1]
80
81  ##
82  #   Solve the linear system
83  ##
84  def Jacobi_Iteration(A,B,C,f,x0,epst):
85      if(sol_type==1):                    # direct solver
86          from Gauss_Elim_File import TomAlgSolver
87          ## ----- ##
88          # HERE IS THE CALL TO GUSSIAN ELIMINATION FUNCTION FROM LAB 1
89          x=TomAlgSolver(A,B,C,f)
90          ## ----- ##
91      elif(sol_type==2):                  # Jacobi iteration
92          x0=np.zeros(N)
93          ## ----- ##
94          # HERE IS THE CALL TO THE jacobi method FUNCTION THAT YOU NEED TO WRITE
95          [x,rnk,iter]=Jacobi_Iteration()
96          ## ----- ##
97          print("Method converged in ",iter," iterations")
98          print("Norm of the final residual %.6E" % rnk)
99      else:
100         print("Invalid solver type")
101
102  ##
103  #   Plot the solution
104  ##
105  xx=np.arange(0,1.0001,h)              # finite element point
106  plt.plot(xx,x,'b',linewidth=1.5)       # plot the solution
107  plt.xlabel("x")                        # label x-axis
108  plt.ylabel("yh(x)")                    # label y-axis
109
110  plt.title('Finite element solution')    # figure title
111  plt.xlim((-0.05,1.05))                 # x-axis limits
112  plt.ylim((min(x)-0.5,max(x)+0.5))      # y-axis limits
```

4. Assembling the linear system to be solved later

5. Using direct (Thomas Algorithm) or iterative (Jacobi Iteration) methods to solve the previously assembled linear system.

2 Task 2

```
1  import numpy as np
2  import sys
3  import matplotlib.pyplot as plt
4
5  #   Input the number of finite elements and boundary conditions
6  ##
7  Ne = int(input("Enter the number of elements: ")) #   number of finite elements
8  y0 = np.zeros(1)
9  y1 = np.zeros(1)
10 y0[0] = input("Input the BC at x=0: ") #   BC at x=0
11 y1[0] = input("Input the BC at x=1: ") #   BC at x=1
12 print("Input the quadrature rule type:") #   quadrature rule type
13 print("  1. Trapezoidal rule")
14 print("  2. Simpson's rule")
15 quad_type=int(input())
16 ##
17 #   Input the solver type
18 ##
19 print("Input the solver type:")
20 print("  1. Direct solver (Gaussian elimination)")
21 print("  2. Jacobi iteration")
22 sol_type=int(input())
23 if int(sol_type==2):
24     epst = float(input("Input the solution tolerance: "))
25
26 ##
27 #   Calculate the number of nodes and the element size
28 ##
29 N = Ne + 1 #   number of nodes
30 h = 1 / Ne #   element size
31 ##
32 #   Initialize the global matrix and the right-hand side vector
33 ##
34 A = np.zeros(N) #   coefficient matrix main diagonal
35 B = np.zeros(N - 1) #   coefficient matrix sub-diagonal
36 C = np.zeros(N - 1) #   coefficient matrix super-diagonal
37 f = np.zeros(N) #   right-hand side vector
38 ##
```

```

39 # Assemble the element matrices and right-hand sides
40 ##
41 #task4 # Change this depending on what task is being completed
42 def func(x):
43     return -12*x**2 + 18*x + 6
44 #task 3
45 #def func(x):
46 #    return (np.pi**2)*np.sin(np.pi*x)
47
48 def Numerical_Quadrature(e,i,quad_type,h):
49     if quad_type == 1:
50         f_i = (h/2) * func((e*1+i)*h)
51         return f_i
52
53
54 for e in range(Ne): #e is element no. # loop over finite elements
55     Ae = np.zeros([2,2]) # element coefficient matrix
56     fe = np.zeros([2]) # element right-hand side
57     for i in range(2): # loop over the nodes in an element
58         ## ----- ##
59         ## HERE IS THE CALL TO THE NUMERICAL QUADRATURE FUNCTION THAT YOU NEED TO WRITE ##
60         fe[i]=Numerical_Quadrature(e,i,quad_type,h); # elelemt right-hand side vector contribution
61         ## ----- ##
62         for j in range(2): # loop over nodes in an element
63             Ae[i,j]=(-1)**(i+j)/h # element matrix contribution
64     ##
65     # Impose the boundary conditions
66     ##
67     if e==0: # the first element at x=0
68         Ae[0,0]=1
69         Ae[0,1]=0
70         fe[0]=y0[0]; # y=y0 at x=0;
71         fe[1]=fe[1]-Ae[1,0]*y0[0] # modify the second equation
72         Ae[1,0]=0
73     if e==Ne-1: # the last element at x=1
74         Ae[1,1]=1
75         Ae[1,0]=0
76         fe[1]=y1[0] # y=y1 at x=1

```

```

77         fe[0]=fe[0]-Ae[0,1]*y1[0]          # modify the second equation
78         Ae[0,1]=0
79     ##
80     #   Transfer the elemnt matrix and the element right-hand side to global data structures
81     ##
82     A[e]=A[e]+Ae[0,0]                      # diagonal element Ae[0,0]
83     A[e+1]=A[e+1]+Ae[1,1]                  # diagonal element Ae[1,1]
84     B[e]=Ae[1,0]                          # sub diagonal element Ae[1,0]
85     C[e]=Ae[0,1]                          # super diagonal element Ae[0,1]
86     f[e]=f[e]+fe[0]                       # rhs element fe[0]
87     f[e+1]=f[e+1]+fe[1]                   # rhs element fe[1]
88     ##
89     #   Solve the linear system
90     ##
91     def Jacobi_Iteration(A,B,C,f,x0,epst):
92         n = len(A)
93         x = np.copy(x0)
94         x_new = np.zeros_like(x)
95         ResidNorm = np.inf
96         iter_num = 0
97
98         diag = A                          # main diagonal
99         sup_diag = B                      # super-diagonal
100        sub_diag = C                      # sub-diagonal
101        # Creating the tridiagonal matrix
102        T = np.diag(diag) + np.diag(sup_diag, 1) + np.diag(sub_diag, -1)
103        B = np.append(B, [0]) # Appending 0 to the end of array B
104        C = np.insert(C, 0, [0]) # Inserting 0 at the beginning of array C
105        while ResidNorm > epst:
106            for i in range(n):
107                sum_Bx = B[i-1] * x[i-1] if i > 0 else 0
108                sum_Cx = C[i+1] * x[i+1] if i < n - 1 else 0
109                x_new[i] = (f[i] - sum_Bx - sum_Cx) / A[i]
110                ResidNorm = np.linalg.norm(f - T @ x_new)
111                x[:] = x_new[:]
112                iter_num += 1
113        return x, ResidNorm, iter_num
114

```

```

115 if(sol_type==1):      # direct solver
116     from Gauss_Elim_File import TomAlgSolver
117     ## ----- ##
118     ## HERE IS THE CALL TO GUSSIAN ELIMINATION FUNCTION FROM LAB 1
119     x=TomAlgSolver(A,B,C,f)
120     ## ----- ##
121 elif(sol_type==2):      # Jacobi iteration
122     x0=np.zeros(N)
123     ## ----- ##
124     ## HERE IS THE CALL TO THE jacobi method FUNCTION THAT YOU NEED TO WRITE
125     [x,rnk,iter]=Jacobi_Iteration(A,B,C,f,x0,epst)
126     ## ----- ##
127     print("Method converged in ",iter," iterations")
128     print("Norm of the final residual %.6E" % rnk)
129 else:
130     print("Invalid solver type")
131 ##
132 # Plot the solution
133 ##
134 xx=np.arange(0,1.0001,h)      # finite element point
135 x_E = np.zeros(len(x))
136 count = 0
137 for p in xx:
138     x_E[count]=np.sin(np.pi*xx[count])
139     count += 1
140
141 inf_norm_diff = np.linalg.norm(x_E-x, np.inf)
142 plt.plot(xx,x,'b',linewidth=1.5)      # plot the solution
143 plt.xlabel("x")      # label x-axis
144 plt.ylabel("yh(x)")      # label y-axis
145
146 plt.title('Finite element solution')      # figure title
147 plt.xlim((-0.05,1.05))      # x-axis limits
148 plt.ylim((min(x)-0.5,max(x)+0.5))      # y-axis limits

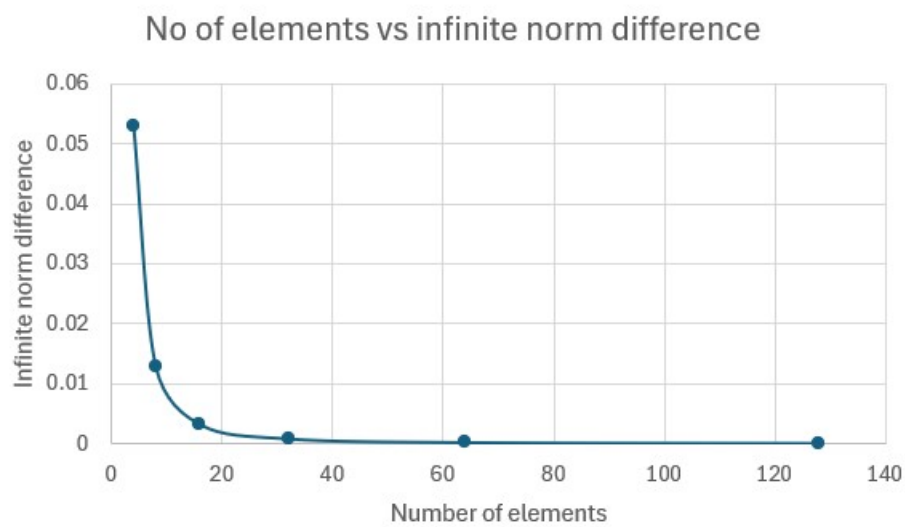
```

```

1 import numpy as np
2
3 def TomAlgSolver(A,B,C,f):
4
5     n = len(A)
6     for i in range(1,n):      #forward substitution
7         A[i] -= (B[i-1] / A[i-1]) * C[i-1]
8         f[i] -= (B[i-1] / A[i-1]) * f[i-1]
9     x = np.ones_like(A)
10    x[-1] = f[-1] / A[-1]
11    for j in range(n-2, -1, -1):      #back substitution
12        x[j] = (f[j] - C[j] * x[j+1]) / A[j]
13
14    return x

```

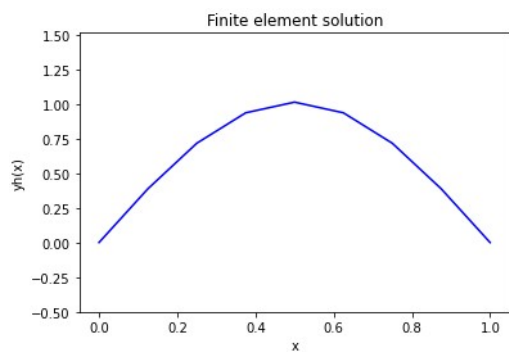
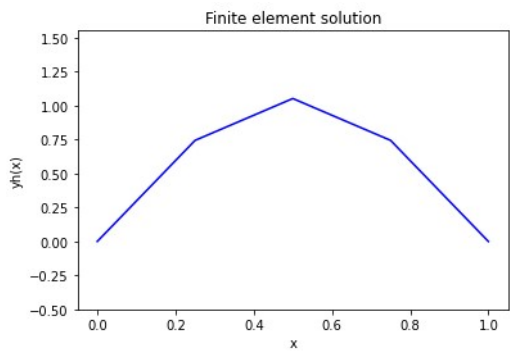

3 Task 3

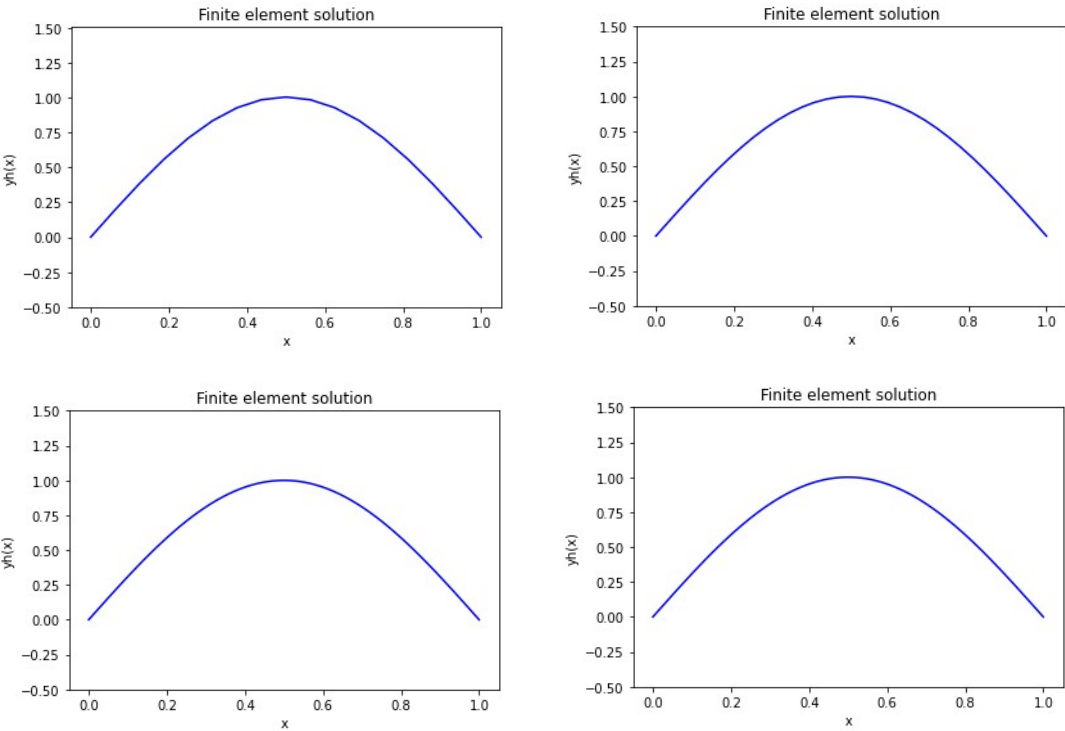


Number of elements	Infinite norm difference
4	0.053029288
8	0.012950747
16	0.003218964
32	0.000803578
64	0.000200822
128	5.02E-05

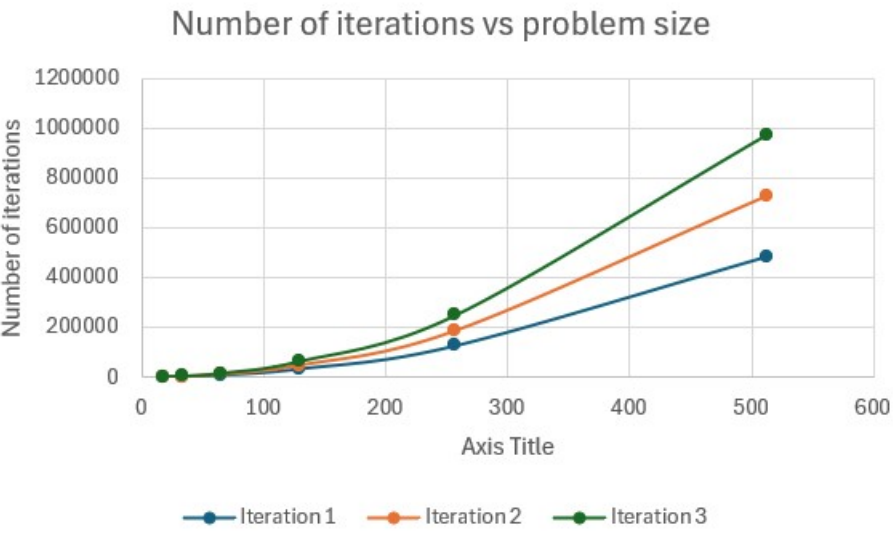
As shown in the figures below, as the finite element mesh is refined, the accuracy increases because the graph becomes a smoother curve with every increase in number of elements. This occurs at an exponential rate, ie the accuracy increases with problem size exponentially.

As can be seen in the table and graph above, the solution error decreases as the problem size increases. This also occurs in an exponential manner. As $N_e \rightarrow \infty$, we can predict that the solution error will follow asymptotic behaviour, due to the shape of the graph so far.





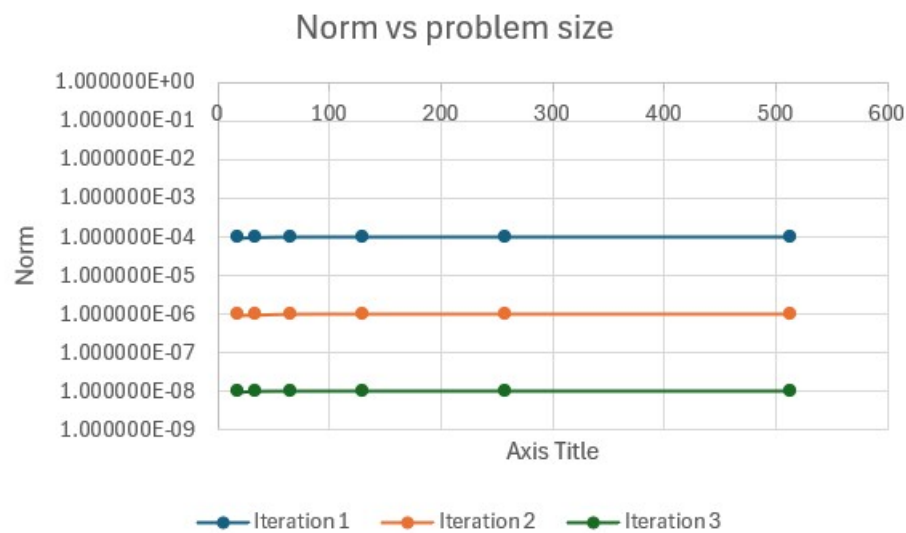
4 Task 4



Problem size	Iteration 1	Iteration 2	Iteration 3
17	561	798	1036
33	2182	3136	4090
65	8448	12269	16090
129	32651	47939	63227
257	126011	187168	248330
513	485643	730275	975095

The graph above shows the number of iterations vs problem size for three different iterations of tolerances (10^{-4} , 10^{-6} , 10^{-8}) for the blue, orange and green lines respectively. All three show the trend that as problem size increases, so does the number of iterations. They all increase at an exponential rate, with the 10^{-8} tolerance increasing the most for a given problem size.

From this data, we can conclude that as problem size increases, the number of iterations will increase exponentially and the rate of this exponential change will increase with decreasing tolerance. Essentially, a smaller tolerance leads to a quicker exponential increase for problem size vs number of iterations.



Problem size	Iteration 1	Iteration 2	Iteration 3
17	9.876187E-05	9.945186E-07	9.822299E-09
33	9.953904E-05	9.956926E-07	9.960023E-09
65	9.995814E-05	9.994158E-07	9.992859E-09
129	9.999230E-05	9.999419E-07	9.998690E-09
257	9.999845E-05	9.999716E-07	9.999736E-09
513	9.999955E-05	9.999833E-07	9.999929E-09

In the graph above, a logarithmic scale was used for the y axis in order to show the smaller values more clearly. Due to the large differences in scale for all three iterations of tolerance, the trend is not as clear on the graph.

Therefore, looking at the table above for norm vs problem size, we can see that as problem size increases, the norm also increases. This is due to the fact that a larger problem size in turn creates a much larger matrix. This requires solving more equations to get the solution which could lead to an amplification in errors. A larger error corresponds to a larger infinity norm, which highlights a smaller result from the Jacobi iteration for a larger problem size.

In conclusion, as problem size increases, the infinity norm increases exponentially at a decreasing rate. This is the same trend for all three tolerances, with the smallest tolerance (10^{-8}) resulting in the smallest infinite norm.